Response to the jury

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang (2362162), M.Sc.A., Electrical Engineering, Polytechnique Montréal
Supervisor: Antoine Lesage-Landry

October 19, 2026

To Sébastien Le Digabel (president), Antoine Lesage-Landry (supervisor) and Antoine Legrain (member of the jury).

Message au jury

Bonjour chers membres du jury,

Je voulais tout d'abord vous remercier pour vos commentaires sur ma thèse. Après avoir reçu de fortes objections sur la qualité des résultats, j'ai pris le temps de les rendre plus complets, et j'ai aussi découvert quelques erreurs dans les affirmations que j'avais faites sur les contributions.

Premièrement, il y a maintenant quatre problèmes qui sont plus lents. J'ai ajouté une section dans la thèse pour les décrire (annexe A.2.1). Trois des quatre problèmes peuvent être beaucoup plus rapides. Deux d'entre eux le deviennent avec des changements dans le « frontend » (par frontend, je fais référence à la manipulation des arbres d'expression avant le calcul des dérivées ; voir les pull requests #3524 et #3570). Par exemple, pouvoir détecter que `D @ ones` (où `D` est une matrice quelconque) équivaut à `D.sum(axis=1)`, la somme sur les colonnes, est un grand avantage. Le troisième, ConvexPlasticity, le devient avec un test de densité (#3571). Ces changements ne règlent pas tout : une partie du ralentissement vient du moteur lui-même, en particulier de la mémoire de la disposition globale (question 5).

Deuxièmement, j'ai utilisé un ordinateur portable plus récent (MacBook M2 de 2022, avec Apple Accelerate) pour cette série de résultats. Pour la première canonicalisation, je pense que les ratios et les temps sont assez similaires. Mais pour les temps de re-résolution paramétrique, la situation a changé : maintenant, seulement deux des problèmes sont plus rapides. J'ai modifié les affirmations de ma conclusion et de la discussion des résultats pour dire seulement qu'il y a des bénéfices, mais qu'il ne remplace pas encore le tenseur pour les re-résolutions.

Troisièmement, le problème UnconstrainedQP n'était pas réellement 100 fois plus rapide. C'était dû à un changement de frontend (#3524) que j'avais appliqué sur la branche du backend diffengine, mais pas sur celles des autres. En l'appliquant à tous, cet avantage disparaît. J'avoue que c'est une grave erreur de ma part, et je l'ai corrigée. Je n'avais pas fait attention aux changements et aux versions du code que j'exécutais pour mes résultats, et surtout, j'avais oublié que ce changement pouvait aussi s'appliquer aux autres méthodes. J'ai fait de mon mieux pour faire preuve de diligence raisonnable, afin que les résultats, cette fois-ci, n'aient pas de problème semblable.

À part ces trois points, qui sont les changements les plus sérieux et les plus importants, j'ai aussi fait les études d'ablation et exécuté plus d'instances de benchmark, avec des mesures de signification statistique, ce qui répond à tous vos commentaires.

Le reste de ce document a été rédigé par Claude à partir de beaucoup de mes indications, surtout pour les réponses à vos questions.

1 Summary of corrections

I have made all the modifications required by the jury and by Antoine Legrain's examiner report, as well as all the minor corrections of Sébastien Le Digabel and of the appendix of the report. The minor corrections are not listed here. All changes since the version sent to the jury are marked in `Document-diff.pdf`, with added text in blue and deleted text in red. Section and page numbers below refer to the final version.

Table 1 lists the required modifications. "J" refers to the jury's requirements after the defence, and (i)–(vii) to the items of the synthesis of the examiner report.

Table 1: Required modifications and where they are made.

Request	Change	Where
J1a: statistical significance	Every timed comparison of 20 runs is tested with a two-sided Mann–Whitney test, Holm-adjusted over the problems of the comparison. Significant ratios are starred in the tables.	Ch. 4 intro and §4.1, p. 52; Tables 4.1, 4.2, A.6, A.7
J1b, (ii): ablation	No dense blocks on the first canonicalization (25 problems, with memory). The split is measured by the first canonicalization and re-solve times of Table 4.2 (see Summary).	§4.1, p. 56; App. A.4, p. 77
J1c: more instances	Each run draws new parameter values from its own seed; the problems are also scaled in size and in density (J1d).	Ch. 4 intro, p. 52; App. A.1, A.5
J1d: sensitivity to size and sparsity	All 25 problems at three sizes, ten families at five sizes, five problems at five densities.	§4.1, p. 56; App. A.5, pp. 79–81
J1e, (iii): non-DPP problems	Re-solves of two families in DPP, non-DPP and parameter-free variants, and of ConvexPlasticity, the suite's one non-DPP parametric problem.	§4.2, p. 56; App. A.3.1, pp. 76–77
J2, (i): related literature and DNLP	Matrix-free modelling, CVXcanon, CVXGEN, Convex.jl 0.16, Pyomo's persistent interfaces, block-sparse antecedents (finite elements, Ceres, g2o), and sparse factorization as the origin of the symbolic/numeric split. §3.1 separates the thesis's own contribution from the DNLP work.	§2.1.3, §2.2, §2.3.1, §2.3.2, §3.1, §3.2
J3: declaration of AI use	New front-matter page.	p. vii

Request	Change	Where
(iv): repeated measurements	20 runs, each in a fresh process; tables give mean ± standard deviation.	Ch. 4 intro, p. 52; Table 4.1, App. A
(v): dimensions of the instances	m, n, nnz(A), nnz(P), parameter size and DPP status of every problem.	App. A, Tables A.1–A.2
(vi): time by phase	§4.1 names the four slower problems and their two causes; App. A.2.1 gives the formulation, the share of each phase and the mechanism for each.	§4.1, p. 55; App. A.2.1, Table A.4
(vii): correctness check	The data are checked against the CPP backend to a relative tolerance of 10^{-9}, at the first canonicalization and after a parameter update.	Ch. 4 intro, p. 52

A change of baseline. The version sent to the jury reported a 2.8× geometric-mean speedup. Two things changed since. All experiments were rerun on an Apple M2 with current versions. And that comparison gave `diffengine` CVXPY PR #3524, which reassociates chains of matrix products ending in a constant, while the other backends were timed on stock CVXPY. PR #3524 benefits every backend, so all four are now timed without it. `diffengine` is the fastest of the four on 21 of the 25 problems, significantly on all 21, with a geometric-mean ratio of 0.43 to the best CVXPY backend, about 2.3×. UnconstrainedQP is now 11.8× slower (§4.1 explains why, and gives the 16 ms it takes with PR #3524).

2 Replies to the defence questions

Q1. *Why materialize dense blocks rather than leave them as operators, as in the matrix-free approach of Diamond and Boyd? When is each approach preferable?*

The conic solvers that CVXPY calls by default factorize a matrix built from A and P, so they need A and P explicitly. Matrix-free modelling pays off for very large problems built on fast transforms and solved by first-order methods. A permuted dense block lies between the two: it is a materialized matrix, but it keeps the structure of the operator, namely its selectors and its dense block. §2.2 now makes this comparison.

Q2. *If the permuted dense representations are disabled, what fraction of the speedup remains?*

Without the dense blocks, `diffengine` is 2.17× faster than the best CVXPY backend in geometric mean, against 2.33× with them. The difference between the two, 1.07×, is not significant over the suite (Wilcoxon signed-rank, $p = 0.07$). Per problem the effect is large in both directions. On six problems with dense data, the dense blocks take the engine from about 3 to about 7× faster than CVXPY. On UnconstrainedQP they take it from 4× faster to 11.8× slower, and on ConvexPlasticity from 1.4 to 6.9× slower (§4.1, App. A.4, Table A.7, which now gives both ratios to CVXPY). The split plays no part in a first canonicalization, since each pass runs once. We do not attribute the remaining speedup to one component: the engine is one design, and only the dense blocks and the split have switches that can be

turned off.

Q3. *How does re-solving behave on a non-DPP parametric problem?*

On the two families with a product of two parameters (`lasso` and `factor_cov`), the cached re-solve is 12 to 13× faster than CVXPY, which has to recompile. ConvexPlasticity, the suite's only natural non-DPP parametric problem, re-solves in 204 ms against 42 ms for CVXPY. The split is not the cause: 88 % of that re-solve assembles the dense 3000×3000 block of P, built from a diagonal given as a dense array. Without dense blocks, the same cached re-solve takes 2.3 ms, 18× faster than CVXPY. The split therefore carries over to a real non-DPP problem; what fails there is the dense storage of a constant that is sparse in fact (§4.2, App. A.3.1).

Q4. *Why not re-evaluate only the subtrees that depend on the updated parameters?*

This is now implemented in the measured engine (SparseDiffEngine #125): after a parameter update, the numeric pass re-evaluates only the subtrees that contain a parameter (§3.4.1). With it, the numeric pass takes 3 to 20 % of a re-solve on the five problems where the tensor is faster. The rest is the conversion to compressed sparse columns and to the cone order, which is now the future-work item (§5.2).

Q5. *When does the global layout become a handicap relative to CVXPY's local layout, in particular for memory?*

In the global layout, each node also keeps work arrays sized by the total number of variables n, whatever variables its own subtree contains. Their memory is therefore $O(\text{nodes} \times n)$ integers, and with many variables and many nodes the local layout is better; this is now stated in the Limitations (§5.1). OptimalAdvertising shows it in time: each of its 250 minimum terms has a Jacobian over all 250 000 variables, and the symbolic pass visits every column (App. A.2.1). Without dense blocks, its peak memory is 945 MB, against 173 to 197 MB for the CVXPY backends; that difference is the per-node cost.

With dense blocks it reaches 6 GB, and most of the difference comes from the aggregate constraints $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ and $\mathbf{D}\mathbf{1} \geq \mathbf{c}$. Canonicalized alone, $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ takes 4 GB with dense blocks and 112 MB without. Its Jacobian is stored as 1000 dense blocks of 250 entries, but each block also keeps an inverse map of its columns with one integer per variable of the problem, 250 000 of them. This is the cost of the global layout again, paid once per dense block rather than once per node. Storing these maps sparsely, or sharing one workspace across the blocks, should remove most of it.

In OptimalAdvertising, both constraints are sums written as products with a vector of ones. A small change to the CVXPY converter, submitted as pull request #3570, lowers such products to the engine's sum, which builds no dense blocks. On OptimalAdvertising it reduces the peak memory from 6.1 GB to 1.4 GB and the canonicalization time from 1.10 s to 0.46 s (App. A.2.1). These are medians of three runs on CVXPY master, outside the measurements of Chapter 4, which are unchanged. The remaining overhead is the per-node cost above, and a weighted sum $\mathbf{D}\mathbf{w}$ with a general dense $\mathbf{w}$ still needs the fix in the engine.

Q6. *What part of SparseDiffEngine pre-existed the thesis or was developed for DNLP, and*

what part is specific to this work?

SparseDiffEngine is joint work with Daniel Cederberg and serves both uses. DNLP differentiates nonlinear atoms at every iterate; this thesis evaluates the affine atoms and the quadratic form to obtain the problem data. Specific to the thesis are the `diffengine` backend, its re-solve path and the benchmarks of Chapter 4 (§1.5, §3.1).

Q7. *What is the strategy for BLAS and binary distribution of sparsediffpy?*

cvxcore is a C++ extension, built with a copy of Eigen, that lives inside the CVXPY repository, so CVXPY has to compile and ship it. sparsediffpy is a separate package: CVXPY only declares it as a dependency and calls it, and the CVXPY side of the backend is much simpler. sparsediffpy is still compiled, in C, and its own wheels are built for Linux, macOS and Windows and published on PyPI. They link Accelerate on macOS and bundle OpenBLAS on Linux and Windows, so users install nothing beyond the package.

3 Summary of the new results

Methodology. Every timed comparison is repeated 20 times, each run in a fresh process, on an Apple M2 (24 GB, macOS 26.5.1) with Accelerate, single-threaded, and tested with a two-sided Mann–Whitney test, Holm-adjusted over the problems (Ch. 4 intro). The sensitivity points of App. A.5 are medians of five runs, reported without a test.

First canonicalization. `diffengine` is the fastest backend on 21 of the 25 problems, with a geometric-mean ratio of 0.43 to the best CVXPY backend. Over the suite, that ratio has a bootstrap 95 % confidence interval of 0.28–0.70, and a Wilcoxon signed-rank test on the log-ratios gives $p = 0.003$. On every problem the runs of the two backends do not overlap, so every adjusted p-value is 1.7×10^{-6}, the smallest attainable with 20 runs (§4.1, Table 4.1).

Ablation. Without dense blocks, the first canonicalization is 1.07× slower in geometric mean (confidence interval 0.71–1.47, $p = 0.07$), and peak memory is 0.85×: a dense block stores every entry, including the zeros of a constant that is sparse in fact (App. A.4, Table A.7). The split needs no separate switch. Without it, a re-solve repeats the conversion and the symbolic pass of the first canonicalization, so it cannot be faster, and Table 4.2 already gives the first canonicalization and the re-solve of every DPP problem. The parameter-to-data tensor is a different mechanism, which can only be compared with the split on re-solves, as Table 4.2 does.

Non-DPP problems. See Q3 above (App. A.3.1, Table A.6).

The four slower problems. App. A.2.1 and Table A.4 give the phase shares and the mechanism for each.

- UnconstrainedQP (98 % numeric pass): a chain of matrix products is evaluated away from its constant and fills a dense Jacobian; CVXPY PR #3524 removes it (16 ms).
- OptimalAdvertising (90 % symbolic pass): the global layout, as in Q5; PR #3570 removes the dense blocks of its aggregate constraints (App. A.2.1).
- ConvexPlasticity: a diagonal given as a dense array becomes a dense block, which PR #3571 stores sparse (App. A.2.1).
- SDPSegfault1132: three dense blocks of 49.5 million entries cancel in a sum with 1.01 million nonzeros, and `diffengine` carries the cancelled entries through both passes. The cancellation depends on the values of the data, so it cannot be seen before the blocks are built.

Sensitivity to size and density. On seven of the ten families measured at five sizes, the ratio stays between 0.13 and 0.22 over a factor of 16 in size. On all 25 problems at three sizes, it stays within 0.04 of its value at the nominal size on 18 of the 21 problems where `diffengine` is faster. The four slower problems stay slower at every size they complete. As the density falls, the advantage shrinks, from a ratio of 0.14–0.17 on dense data to 0.40–2.56 at 1 % density, and above 20 % density `diffengine` is faster on every problem (App. A.5, Tables A.8–A.10).

What the results imply. The dense blocks help when the data is dense and hurt when a constant that is sparse in fact is stored dense. The backend already stores a constant factor of a matrix product sparse below 5 % nonzeros, but the quadratic objective had no such test. Pull request #3571 applies it there. On ConvexPlasticity, whose diagonal is given as a dense array, it makes canonicalization 3 times faster (194 against 566 ms, medians of 10 runs on CVXPY master, outside the measurements of Chapter 4) with the same problem data, as App. A.2.1 now notes. Most of the rest of its slowdown, like the memory of OptimalAdvertising (Q5), comes from the inverse column maps of the dense blocks in the engine. Setting the threshold from the cost of the kernels, for the slowdowns at low density, remains future work (§5.2).